Assignment 1, Talha Yusuf, 2504011

Dataset : Text classification (News Groups)

Part 1 :

First we selected the Text classification (News Groups) dataset to work.

Then loaded the dataset and found out the structure of the data and labels for the first 5 samples. It is a subset of 20 Newsgroups containing two categories: rec.autos and comp.graphics.

The training samples: 1,178 and test samples: 785

Sample

```
Training data size: 1178
Categories: ['comp.graphics', 'rec.autos']
First 5 samples (first 200 characters each):
Sample 1: From: dave.mikelson@almac.co.uk (Dave Mikelson)
Subject: Re: PCX
Distribution: world
Organization: Almac BBS Ltd. +44 (0)324 665371
Reply-To: dave.mikelson@almac.co.uk (Dave Mikelson)
Lines: 22
```

Part 2:

for part 2 we applied TF-IDF for traditional word embeddings. Then we wanted to implement Word2Vec embedding as the pre trained model. So, we installed Gensim to load pre-trained Word2Vec embeddings. Minimal cleaning was performed: Text was kept in lowercase (handled internally by vectorizers).

No URLs, emails, or special character removal was explicitly needed because the dataset is relatively clean. Texts were then transformed into numeric features for modeling:

TF-IDF: Sparse vectors representing term importance, using unigrams and bigrams.

Word2Vec: Dense 300-dimensional embeddings, computed as the average of all pre-trained word vectors in each document.

Part 3:

Lastly we applied Logistic Regression classifier using traditional methods and pre-trained

embeddings.

Logistic Regression works with both sparse TF-IDF vectors and dense Word2Vec embeddings. Word2Vec embeddings rely on pre-trained vocabulary; any words not in the vocabulary were represented as zero vectors.

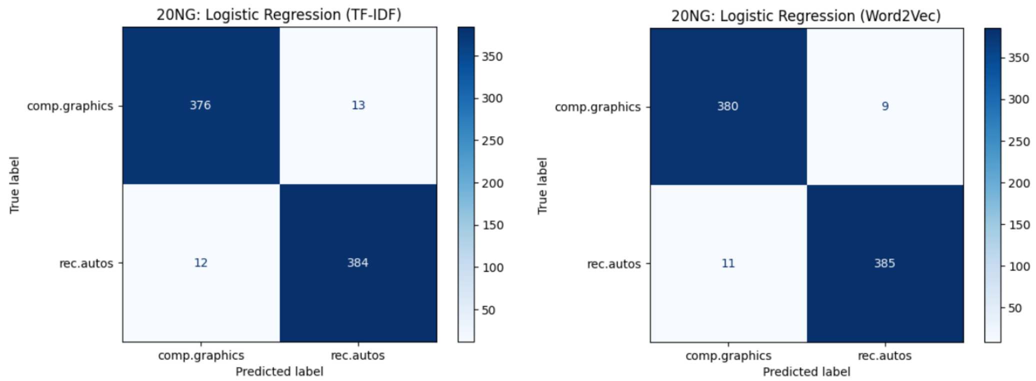
Results:

Embedding	*Dataset*	*Accuracy*	*Precision*	*Recall*	*F1-score*
TF-IDF	*comp.graphics*	0.97	0.95	0.99	0.97
	rec.autos	0.97	0.99	0.95	0.97
Word2Vec	*comp.graphics*	0.95	0.96	0.95	0.96
	rec.autos	0.95	0.95	0.96	0.95

Accuracy is consistent across both categories (for TF-IDF Embedding), at 97% for both comp.graphics and rec.autos. Precision and recall are balanced for each category, with rec.autos achieving a slightly higher precision of 99%, and comp.graphics showing a slightly higher recall of 99%. The F1-score for both categories is also close, with a value of 97% for both.

Accuracy improves slightly with Word2Vec, reaching 95% for both categories. Precision is higher for comp.graphics (0.96) compared to rec.autos (0.95). Recall is particularly strong for rec.autos (0.99) but slightly lower for comp.graphics (0.95). The F1-score is higher for both categories when using Word2Vec, reaching 0.96 for comp.graphics and 0.95 for rec.autos.

Confusion Matrix :



Dataset : Sentiment Analysis (IMDB reviews)

Part 1 :

To work with this dataset, we downloaded the .tar file and uploaded it to files of google colab and used the tar command to extract the dataset. Then we took some sample positive and negative reviews, it gave me a manageable dataset.

```
Training samples: 25000
Test samples: 25000
First training review label: 1
First training review text:
What an overlooked 80's soundtrack. I imagine John Travolta sang some o
```

Part 2:

for part 2 we applied TF-IDF for traditional word embeddings. Then we wanted to implement Word2Vec embedding as the pre trained model. So, we used the preinstalled Gensim to load pre-trained Word2Vec embeddings.

Minimal preprocessing, similar to 20 Newsgroups, we kept text in original lowercase. Stopwords removal was handled by the TF-IDF vectorizer. TF-IDF: Sparse matrix representation of unigrams and bigrams (min_df=5, max_df=0.9). Word2Vec: Dense vectors computed as average of pre-trained 300D word vectors.

Output:

TF-IDF training matrix shape: (25000, 81097)

TF-IDF test matrix shape: (25000, 81097)

Word2Vec training matrix shape: (25000, 300)

Part 3:

Lastly we apply Logistic Regression classifier using traditional methods and pre-trained embeddings. MDB Reviews: lowercase conversion, TF-IDF and Word2Vec embeddings. For Logistic Regression modeling, Stopwords were removed automatically by TF-IDF, and documents with words missing from the Word2Vec vocabulary were represented as zero vectors. These preprocessing steps prepared the data.

Results:

Embedding	***Dataset***	***Accuracy***	***Precision***	***Recall***	***F1-score***
TF-IDF	*0*	0.88	0.88	0.88	0.88
	1	0.88	0.88	0.88	0.88
Word2Vec	*0*	0.83	0.82	0.84	0.83
	1	0.83	0.83	0.82	0.83

TF-IDF embeddings achieved better performance across all metrics (accuracy, precision, recall, and F1-score), with a perfect balance for both classes (0 and 1) Accuracy 88%. Word2Vec embeddings performed slightly lower than TF-IDF in terms of accuracy, and had a slightly less balanced performance, especially for the Positive class. Word2Vec's strength lies in its ability to generalize well across words not explicitly in the training data, but it seems that TF-IDF is better suited for sentiment classification in this case, as it focuses on the most discriminative terms in the dataset.

Confusion Matrix :

```
Training samples: 25000, Test samples: 25000
```